

Homework Exercises 3

Dr. Sarvar Abdullaev
s.abdullaev@inha.uz

September 22, 2022

After completing all lab exercises below, zip all your solution files into a single archive with your student ID (e.g. u210023.zip) and upload it to eClass before the deadline.

Note, you do not have to submit other lab materials such as lab slides or examples.

1 Representing Data

1.1 Representing Text

1. This message is written in ASCII encoding and represented with hexadecimal digits. Write it with English letters. Make sure that you write letters with correct case and do not skip any punctuation characters. Save your answer in `p1-1.txt` file.

```
49 20 6C 6F 76 65 20 43 6F 6D 70 75 74 65 72 20 53 63 69 65 6E 63 65 21
```

2. This message is written in UTF-8 encoding and represented with hexadecimal digits. Write it with English letters and corresponding emoticons. Make sure that you write letters with correct case and do not skip any punctuation characters. Save your answer in `p1-2.txt` file.

```
48 65 6c 6c 6f 20 57 6f 72 6c 64 21 20 49 6e 74 72 6f 20 74 6f 20 49 54 20 69 73 20 74
68 65 20 62 65 73 74 20 63 6f 75 72 73 65 20 61 74 20 49 55 54 2e 20 f0 9f 98 80
```

Hint. You can use following Python code to encode/decode text into hex.

```
# -*- coding: utf-8 -*-

from binascii import unhexlify

fromEncoding = 'utf-8'
toEncoding = 'ascii'

str = "Hello World!\U0001f600"
print(str)

hex_encoded = str.encode(fromEncoding, errors='ignore').hex()
print(hex_encoded)

decoded_str = unhexlify(hex_encoded).decode(toEncoding, errors='replace')
print(decoded_str)
```

1.2 Representing Numbers

1. Compute each of the following problems by translating the values into two's complement notation with 5 bits, converting any subtraction problem to an equivalent addition problem, and performing that addition. Check your work by converting your answer to base 10 notation. Check for possible overflows. Save your answer in `p1-3.txt` file.
 - (a) $5 - 1$
 - (b) $12 - 5$
 - (c) $8 - 7$
 - (d) $12 + 5$
 - (e) $5 - 11$
2. Express each of the following values in 8-bit floating point notation, where 1 bit for sign, 3 bits for exponent and 4 bits for mantissa. Check for possible truncations. Save your answer in `p1-4.txt` file.
 - (a) $5\frac{3}{4}$
 - (b) $15\frac{15}{16}$
 - (c) $5\frac{3}{8}$
 - (d) $1\frac{1}{4}$
 - (e) $1\frac{1}{5}$
3. Three 8-bit patterns 10000010, 01101000 and 00000010 are representations of the same value in two's complement, excess and 8-bit floating point notation, but not necessarily in the same order. What is this common value and which pattern is in which notation? Save your answer in `p1-5.txt` file.

2 Python Tasks

Watch **Week 2: Simple Programs** lecture videos on this course. Then complete following Python tasks.

1. Using Newton's method discussed in Week 1 and shown below, first find an iterative update method of your initial guess x_0 finding the cubic root $\sqrt[3]{S}$ of a number S , and then implement an algorithm in Python for calculating the cubic root $\sqrt[3]{S}$ to a precision of $\epsilon = 0.0001$.

$$x_{t+1} = x_t - \frac{f(x_t)}{f'(x_t)} \quad (1)$$

Save this algorithm as `newtons_method.py`.

2. Set $S = 30$, then start your algorithm with following guesses as shown in Table 1. Then, count the number of guesses your algorithm makes before it reaches the necessary precision and stops. Fill the table below.

Initial Guess (x_0)	Number of Iteration
0.5	
1.0	
1.5	
2.0	
2.5	
3.0	
3.5	
4.0	
4.5	
5.0	

Table 1: Number of Iterations per Guess

Plot it on a graph as the function of your initial guess to the number of guesses. What mathematical relationship does it resemble?

As an optional exercise, you may plot a similar relationship between absolute error $e = |\hat{x} - x_0|$ between your initial guess x_0 and your satisfactory solution $\hat{x} = \sqrt[3]{S}$ for some arbitrary S versus the number of guesses your algorithm has made.

- Review following code for finding the square root using bisection search method.

```
x = 25
epsilon = 0.0001
numGuesses = 0
low = 0.0
high = max(1.0, x)
ans = (high + low)/2.0
while abs(ans**2 - x) >= epsilon:
    print('low =', low, 'high =', high, 'ans =', ans)
    numGuesses += 1
    if ans**2 < x:
        low = ans
    else:
        high = ans
    ans = (high + low)/2.0
print('numGuesses =', numGuesses)
print(ans, 'is close to square root of', x)
```

Implement a similar algorithm using bisection search for finding the cubic root of x in Python. Save this algorithm as `bisection_method.py`.

- Run two algorithms you have implemented above for finding the cubic root of different numbers ranging from 10 to 100, and count the number of iterations they make respectively. Fill the Table 2 below.

Input Number (S)	Newton's Method	Bisection Method
10		
20		
30		
40		
50		
60		
70		
80		
90		
100		

Table 2: Number of Iterations per Method

Do you think the results differ significantly?